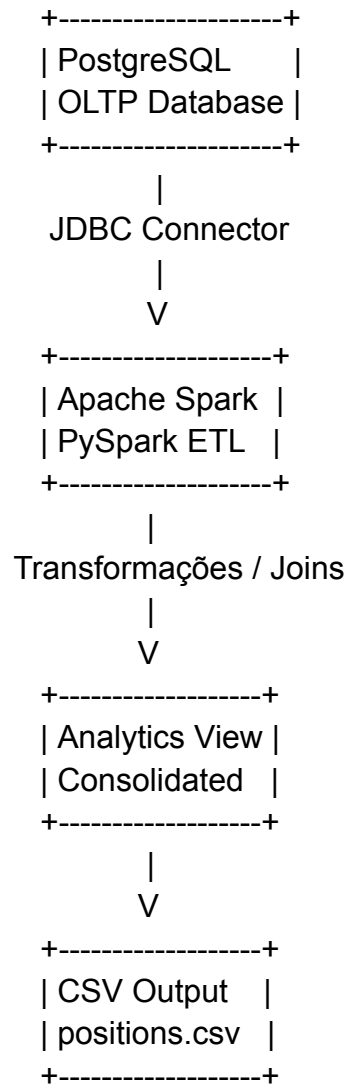


Desenho simples da arquitetura proposta



Stack:

- Banco: PostgreSQL
- Linguagem: Python
- ETL: PySpark
- Container: Docker Compose
- Testes: Pytest

Resumo simples da estrutura do projeto

asset-management-data-engineering/

```
|
|
|— docker-compose.yml
|
|
|— README.md
|
|
|— database/
|  |— init.sql
|  └─ inserts.sql
|
|
|— spark/
|  |— jobs/
|  |  └─ positions_etl.py
|  |
|  |
|  └─ requirements.txt
|
|
|— tests/
|  └─ test_etl.py
|
|
└─ output/
    └─ positions.csv
```

Modelagem do Banco

- **tb_fundos**

```
create table tb_fundos (  
id_fundo SERIAL PRIMARY KEY,  
nome_fundo VARCHAR(100),  
categoria VARCHAR(50),  
patrimonio_liquido NUMERIC);
```

- **tb_emissores**

```
create table tb_emissores (  
id_emissor SERIAL PRIMARY KEY,  
nome_emissor VARCHAR(100),  
setor VARCHAR(50));
```

- **tb_ativos**

```
create table tb_ativos (  
id_ativo SERIAL PRIMARY KEY,  
codigo VARCHAR(20),  
tipo_ativo VARCHAR(50),  
id_emissor INT,  
FOREIGN KEY(id_emissor)  
REFERENCES emissores(id_emissor));
```

- **tb_precos_mercado**

```
create table tb_precos_mercado (  
id_preco SERIAL PRIMARY KEY,  
id_ativo INT,
```

```
data_preco DATE,  
preco NUMERIC,  
FOREIGN KEY(id_ativo)  
REFERENCES ativos(id_ativo));
```

- **tb_operacoes**

```
create table tb_operacoes (  
id_operacao SERIAL PRIMARY KEY,  
id_fundo INT,  
id_ativo INT,  
tipo_operacao VARCHAR(20),  
quantidade NUMERIC,  
data_operacao DATE);
```

- **tb_posicoes (Fato principal)***

```
create table tb_posicoes (  
id_posicao SERIAL PRIMARY KEY,  
id_fundo INT NOT NULL,  
id_ativo INT NOT NULL,  
data_referencia DATE NOT NULL,  
quantidade NUMERIC(18,4),  
preco_medio NUMERIC(18,4),  
valor_financeiro NUMERIC(18,2),  
percentual_pl NUMERIC(10,4));
```

INSERTS de massa fictícia

```
INSERT INTO tb_fundos VALUES
```

```
(1,'Fundo Alpha','Renda Fixa',10000000),
```

```
(2,'Fundo Beta','Multimercado',5000000);
```

```
INSERT INTO tb_emissores VALUES
```

```
(1,'Tesouro Nacional','Governo'),
```

```
(2,'Petrobras','Energia');
```

```
INSERT INTO tb_ativos VALUES
```

```
(1,'TESOURO2028','Titulo Publico',1),
```

```
(2,'PETR4','Ação',2);
```

```
INSERT INTO tb_posicoes VALUES(
```

```
1,1,1,'2026-01-01',1000,120,120000,2.5);
```

ETL Pyspark

Arquivo: spark/jobs/positions_etl.py

Codigo: from pyspark.sql import SparkSession

```
spark = (
```

```
    SparkSession
```

```
        .builder
```

```
        .appName("AssetManagementETL")
```

```
.getOrCreate()
)
jdbc="jdbc:postgresql://postgres:5432/assets"
properties={
  "user":"postgres",
  "password":"postgres",
  "driver":"org.postgresql.Driver"
}
fundos = spark.read.jdbc(
  jdbc,
  "tb_fundos",
  properties=properties
)
ativos = spark.read.jdbc(
  jdbc,
  "tb_ativos",
  properties=properties
)
emissores = spark.read.jdbc(
  jdbc,
  "tb_emissores",
  properties=properties
)
```

```
posicoes = spark.read.jdbc(
    jdbc,
    "tb_posicoes",
    properties=properties
)

# Join das dimensões

resultado = (tb_posicoes
    .join(tb_fundos,
        "id_fundo"
    )
    .join(tb_ativos,
        "id_ativo"
    )
    .join(tb_emissores,
        "id_emissor"
    ))

resultado = (tb_posicoes
    .join(tb_fundos,
        tb_posicoes.id_fundo == tb_fundos.id_fundo,
        "inner")
    .join(tb_ativos,
        tb_posicoes.id_ativo == tb_ativos.id_ativo,
        "inner")
```

```
.join(tb_emissores,
      tb_ativos.id_emissor == tb_emissores.id_emissor,
      "inner"))
resultado_final.write \
.mode("overwrite") \
.option("header",True)\
.csv("/output/positions")
spark.stop()
```

Resultado gerado em CSV

nome_fundo,codigo,tipo_ativo,nome_emissor,data_posicao,quantidade,preco,valor_financeiro

Fundo Alpha,TESOURO2028,Titulo Publico,Tesouro Nacional,2026-01-01,1000,120,120000

Docker Compose

Arquivo YAML:

version: '3'

services:

postgres:

image: postgres:15

environment:

POSTGRES_DB: assets

POSTGRES_USER: postgres

POSTGRES_PASSWORD: postgres

ports:

- "5432:5432"

volumes:

- ./database:/docker-entrypoint-initdb.d

spark:

image: bitnami/spark

depends_on:

- postgres

volumes:

- ./spark/jobs:/jobs

- ./output:/output

Para executar todo ETL é necessário primeiramente rodar em Bash o código (docker compose up -d) e depois o código (spark-submit --packages rg.postgresql:postgresql:42.7.0/jobs /positions_etl.py

Teste unitário

Arquivo: test_etl.py

```
from pyspark.sql import SparkSession
```

```
def test_join():
```

```
    spark = SparkSession.builder \
```

```
        .master("local") \
```

```
        .getOrCreate()
```

```
    data= [ ("Fundo Alpha", "PETR4")]
```

```
df=spark.createDataFrame(  
    data,  
    ["fundo","ativo"]  
)  
assert df.count()==1
```